

Rename `join` and `nest` in `async_scope` proposal

Document #: P3706R0 [[Latest](#)] [[Status](#)]
Date: 2025-05-19
Project: Programming Language C++
Audience: LEWG
Reply-to: Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>
Ville Voutilainen
<ville.voutilainen@gmail.com>

Contents

- [1 Motivation](#)
- [2 Rename `join` member functions](#)
- [3 Rename `nest`](#)
- [4 Proposal](#)
- [5 Wording](#)
- [6 References](#)

§ Abstract

This paper proposes renaming `join` and `nest` methods in `async_scope` proposal [[P3149R9](#)]

§ 1 Motivation

[[P3149R9](#)] provides a one of the important pieces for [[exec](#)] ([[P2300R10](#)] proposal) to complete a picture. The facilities allow to group the asynchronous work by associating it with scopes and then wait at some point for all associated work to be completed. It a bridge from the unstructured concurrency to the structured one with saying that individual pieces of work (possibly on different execution contexts) become the one, combined work. The facilities also support

- associating the work and run it (eager execution)
- associating the work but getting back a sender to run it later (lazy execution)

Despite all the advantages, some names might be arguably misleading because they are either not following the existing practices or might have a different meaning in the same or very close domains: concurrency and parallelism.

[P3149R9] has a section that ruminates about naming. But this discussion never happened in LEWG and/or in reflector.

[P3685R0] started making improvements by proposing `async_scope_token` -> `scope_token` renaming. We want to make it even better with this proposal by renaming `join` and `nest`. There are other candidates to be renamed as well, however the existing names are also fine and authors cannot come up with better alternative, so other than `join` and `nest` the rest is out of scope for this paper.

§ 2 Rename `join` member functions

The name `join` is problematic because we already have `join` in the standard for threads but in a different way. [P3149R9] tells about similarity between `std::thread::join` and `execution::counting_scope::join`, however there is a fundamental difference: `join` for `std::thread` is a blocking call. It blocks the current thread and waits till its completion. `counting_scope::join`, on the other hand, is not blocking. It returns a sender that signals when all the work associated with a scope is completed. Consider the following comparison table:

`thread::join` vs `counting_scope::join`

<code>thread::join</code>	<code>counting_scope::join</code>
<pre>// create the // thread // doing some // work std::thread t1([]{ /* some work */ }); // block a current // thread and // waits for // t1 // completion</pre>	<pre>namespace ex = std::execution; my_thread_pool pool; ex::counting_scope scope; // create sender to run within async scope auto snd = ex::transfer_just(pool.get_scheduler()) ex::then([] { /* some work */ }); // associate snd and launch the work ex::spawn(snd, scope.get_token()); // !!! Has no effect. Returns sender that is // discarded scope.join(); // Users have to call sync_wait explicitly on // the</pre>

<code>thread::join</code>	<code>counting_scope::join</code>
<code>t1.join();</code>	<code>// returned sender by join to get the expected behavior</code> <code>this_thread::sync_wait(scope.join());</code>

As you can see in the example above, `counting_scope::join` does not block as some might expect. It returns a sender that can signal that the all associated work with the scope is completed. Somebody might argue that this is a quality of implementation: implementers can put `[[nodiscard]]` attribute on the function and emit a warning. While this is true it does not eliminate the fact that the name itself it not helpful to recognize such behavior.

In [\[libunifex\]](#) there is a v1 of `async_scope` (the latest one is v2 as far as we can tell) that names this function as `complete`. While this name is better in our opinion, it still does not give any clue that the function itself is non-blocking.

Together with authors of [\[P3149R9\]](#) and some other people we considered a bunch of names. For example, `complete`, `cleanup`, `drain`, `completes`, `completes_when`, `done`, etc.

It seems like people converge to have `when_` prefix, similar to `when_all`. Given that we mentioned a term *completion* several times in this paper, we suggest to rename `join` function to `when_completed` because in our opinion it's much clearer and gives users better understanding of what's going on.

The last line of the code in the column with `counting_scope` looks like this with out proposal:

```
this_thread::sync_wait(scope.when_completed());
```

§ 3 Rename nest

There is a term that is called *nested parallelism*. It means that some piece of work spawns more work from a asynchronous/parallel context.

One of the APIs that creates a nested parallelism (or nested work) in [\[P2300R10\]](#) is `let_value`. It, basically, runs the returned sender from the passed function and waits for its completion to send the computed values further. This returned-by-the-passed-function-sender is a nested work. [\[P3149R9\]](#) itself contains the example with nested parallelism in the example with handling a tree using `spawn_future` and `let_value`.

However, `nest` name itself doesn't imply nested parallelism at all. That's why it's confusing. When users call `nest` they basically want to associate the piece of work with the scope, or attach the piece of work to the scope. The execution of the work is deferred.

The API nest might be also called like that by historical reasons: previously [P3149R9] proposal had nest as one of the required operations on scopes by `async_scope` concept. The current state is that the `async_scope` concept was replaced by the lower level one named `async_scope_token` (hopefully, just `scope_token` after applying [P3685R0]) with requiring `try_associate` and `deassociate` methods (plus `wrap`). Also, v1 `async_scope` in [libunifex] had the `attach` name instead of `nest`.

In our desire to avoid confusion and also taking into account the previous experience we can recommend two names instead of `nest`. It's either `associate` or `attach`. Given that [P3149R9] talks about association a lot, this is the preferred option, unless authors of [P3149R9] think that the similarity to `try_associate` is more confusing than helpful. If this is the case then `attach` is also a good choice.

§ 4 Proposal

- Rename `join` to `when_completed`
- Rename `nest` to `associate`

§ 5 Wording

Since [P3149R9] is still in active LWG review phase the wording for renaming does not make a lot of sense. We will need to work with `async_scope` authors to make sure that all the changes are incorporated from this proposal, if it passes.

§ 6 References

[libunifex] libunifex.

<https://github.com/facebookexperimental/libunifex>

[P2300R10] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2024-06-28. ``std::execution``.

<https://wg21.link/p2300r10>

[P3149R9] Ian Petersen, Jessica Wong; Dietmar Kühl; Ján Ondrušek; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; Ruslan Arutyunyan; 2025-02-18. `async_scope` — Creating scopes for non-sequential concurrency.

<https://wg21.link/p3149r9>

[P3685R0] Robert Leahy. Rename `async_scope_token`.

<https://isocpp.org/files/papers/P3685R0.pdf>